

# VNU-HUS MAT3500 3: Toán rời rạc

## Thuật toán II

Thuật toán đệ quy, thuật toán tham lam

Hoàng Anh Đức

Bộ môn Tin học, Khoa Toán-Cơ-Tin học  
Đại học KHTN, ĐHQG Hà Nội  
[hoanganhduc@hus.edu.vn](mailto:hoanganhduc@hus.edu.vn)



# Nội dung



## Thuật toán II

Hoàng Anh Đức

### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

## Giới thiệu



### Thuật toán II

Hoàng Anh Đức

#### Thuật toán đệ quy

2

##### Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

##### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

##### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

- Định nghĩa theo đệ quy không những có thể áp dụng cho các hàm và tập hợp mà còn cho cả các thuật toán
- Một *thủ tục đệ quy (recursive procedure)* là một thủ tục gọi chính nó
- Một *thuật toán đệ quy (recursive algorithm)* là một thuật toán giải một bài toán bằng cách chuyển về việc giải chính bài toán đó nhưng với đầu vào có kích thước nhỏ hơn
  - *Kỹ thuật chia để trị (divide-and-conquer technique)*: giải một bài toán ban đầu thông qua việc chia nó thành các bài toán nhỏ hơn cùng loại và giải chúng

# Thuật toán đệ quy

## Giới thiệu



### Thuật toán II

Hoàng Anh Đức

Một thuật toán đệ quy thường có cấu trúc như sau:

**Trường hợp cơ sở** Giải trực tiếp bài toán với đầu vào có kích thước đủ nhỏ hoặc đầu vào đặc biệt mà không cần gọi đệ quy

**Chia thành các bài toán con** Với đầu vào không thuộc trường hợp cơ sở, thuật toán chia bài toán thành một hoặc nhiều **bài toán con** có kích thước nhỏ hơn

**Giải các bài toán con** Thuật toán gọi đệ quy chính nó để giải các bài toán con đã được định nghĩa

**Kết hợp các lời giải** Thuật toán kết hợp các lời giải của các bài toán con để tạo ra lời giải cho bài toán ban đầu

### Thuật toán đệ quy

3

#### Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

## Giới thiệu



### Thuật toán II

Hoàng Anh Đức

Chứng minh tính đúng đắn của thuật toán đệ quy bằng quy nạp mạnh

- **Phát biểu điều cần chứng minh:** Một điểm quan trọng là cần *chỉ rõ “thuật toán đúng” nghĩa là gì*
- **Bước cơ sở:** Các trường hợp khi *thuật toán cho ra kết quả một cách trực tiếp* mà không cần thông qua gọi đệ quy chính nó là các trường hợp cần xét trong bước cơ sở
  - Sử dụng mô tả của thuật toán để chỉ ra thuật toán sẽ trả lại gì trong trường hợp cơ sở
  - Chỉ ra rằng giá trị trả lại của thuật toán là đúng
- **Bước quy nạp:** Giả thiết rằng thuật toán đúng cho *mọi đầu vào kích thước nhỏ hơn*. Chỉ ra rằng thuật toán cũng đúng cho đầu vào hiện tại
  - Phát biểu giả thiết quy nạp: Giả sử thuật toán đúng với mọi đầu vào giữa trường hợp cơ sở và các đầu vào có kích thước nhỏ hơn một đơn vị so với đầu vào hiện tại
  - Mô tả cụ thể thuật toán trả lại gì với đầu vào hiện tại dựa trên các lần gọi đệ quy
  - Sử dụng giả thiết quy nạp để thay mỗi lần gọi đệ quy bằng đáp án chính xác. Chỉ ra rằng những điều này dẫn tới đáp án đúng cho trường hợp hiện tại
  - Nếu bạn xét nhiều trường hợp trong thuật toán thì cần thực hiện hai điều trên với từng trường hợp một

### Thuật toán đệ quy

#### Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

## Giới thiệu



Thuật toán II

Hoàng Anh Đức

Phân tích thời gian chạy của thuật toán đệ quy:

### ■ *Thiết lập hệ thức truy hồi*

- Biểu diễn thời gian chạy  $T(n)$  dưới dạng các hàm của thời gian chạy cho các bài toán con

### ■ *Giải hoặc ước lượng hệ thức truy hồi*

- Giải hệ thức truy hồi (đã đề cập trong phần “Quy nạp và Đệ quy”)
  - Đoán nghiệm và chứng minh bằng quy nạp
  - Sử dụng hàm đặc trưng
  - Sử dụng hàm sinh
- Ước lượng hệ thức truy hồi (sẽ đề cập trong phần sau)
  - Định lý thợ (Master Theorem)
  - Cây đệ quy (Recursion Tree)

#### Thuật toán đệ quy

5

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

Tính giai thừa



Thuật toán II

Hoàng Anh Đức

Với mọi số nguyên không âm  $n$

$$0! = 1$$

$$n! = n \cdot (n - 1)! \quad \forall n \geq 1$$

## Thuật toán 1: Tính $n!$

**Input:**  $n$ : số nguyên không âm

**Output:**  $n!$

**procedure** factorial( $n$ ):

**if**  $n = 0$  **then**

**return** 1

**else**

**return**  $n \cdot \text{factorial}(n - 1)$

Thuật toán đệ quy

Giới thiệu

6

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

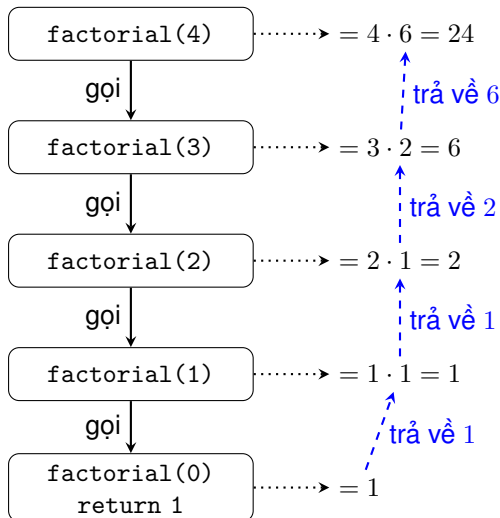
Bài toán Lập lịch

# Thuật toán đệ quy

Tính giai thừa



## Ví dụ 1 (factorial(4))



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

7

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thặng dư

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

Tính giai thừa



Thuật toán II

Hoàng Anh Đức

## Ví dụ 2 (Thuật toán đệ quy tính giai thừa là đúng)

Ta chứng minh tính đúng đắn của Thuật toán 1 bằng quy nạp. Gọi  $\text{factorial}(n)$  là giá trị trả lại bởi Thuật toán 1

- Ta chứng minh  $\text{factorial}(n) = n!$  với mọi  $n \geq 0$
- **Bước cơ sở:** Khi  $n = 0$ ,  $\text{factorial}(n) = 1 = n!$
- **Bước quy nạp:** Giả sử  $\text{factorial}(k) = k!$  với số nguyên  $k \geq 0$  nào đó. Ta chứng minh  $\text{factorial}(k+1) = (k+1)!$ .  
Thật vậy, Thuật toán 1 trả lại  
 $\text{factorial}(k+1) = (k+1) \cdot \text{factorial}(k)$ . Theo giả thiết quy nạp,  $\text{factorial}(k) = k!$ . Do đó,  
 $\text{factorial}(k+1) = (k+1) \cdot k! = (k+1)!$

## Ví dụ 3 (Thời gian chạy của thuật toán đệ quy tính giai thừa)

$$T(n) = \max\{O(1), T(n-1) + O(1)\} + O(1) = T(n-1) + O(1)$$

nghĩa là tồn tại hằng số  $C$  thỏa mãn  $T(n) = T(n-1) + C$ . Suy ra  $T(n) = O(n)$

### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thặng dư

Cây đệ quy

### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

8

37

# Thuật toán đệ quy

Tính lũy thừa



Thuật toán II

Hoàng Anh Đức

Với mọi số thực  $a \neq 0$  và số nguyên không âm  $n$ ,

$$a^0 = 1$$

$$a^n = a \cdot a^{n-1} \quad \forall n \geq 1$$

## Thuật toán 2: Tính $a^n$

**Input:**  $a$ : số thực khác 0,  $n$ : số nguyên không âm

**Output:**  $a^n$

**procedure** power( $a, n$ ):

**if**  $n = 0$  **then**

**return** 1

**else**

**return**  $a \cdot \text{power}(a, n - 1)$

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

## Bài tập 1

Với Thuật toán 2, hãy

(a) Chứng minh tính đúng đắn

(b) Xây dựng hệ thức truy hồi để đánh giá thời gian chạy

# Thuật toán đệ quy

Tính lũy thừa



Thuật toán II

Hoàng Anh Đức

## Chú ý

Điều khó khăn nhất cần vượt qua khi suy nghĩ về các thuật toán đệ quy là sự miễn cưỡng không tin rằng trường hợp nhỏ hơn sẽ được xử lý đúng.

- Ví dụ, giả sử bạn muốn viết một thuật toán đệ quy để tính  $a^n$ , sử dụng các công thức  $a^{2k} = (a^k)^2$  và  $a^{2k+1} = (a^k)^2 \cdot a$ .
- Lệnh gọi đệ quy sẽ xử lý việc tính toán  $a^k$ , vì vậy bạn không cần phải lo lắng về cách máy tính sẽ đệ quy nhiều lần, tất cả đường xuống đến trường hợp cơ sở, để thực hiện điều đó. *Miến là bước đệ quy và trường hợp cơ sở của bạn (ở đây là  $a^0 = 1$ ) là đúng, thuật toán của bạn là đúng*

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

10

37

# Thuật toán đệ quy

## Tìm kiếm tuyến tính



### Thuật toán II

Hoàng Anh Đức

## Thuật toán 3: Tìm kiếm tuyến tính (Linear Search)

**Input:**  $a_1, a_2, \dots, a_n$ : dãy số nguyên,  $i, j, x$ : số nguyên,  
 $1 \leq i \leq j \leq n$

**Output:** Nếu  $x \in \{a_i, a_{i+1}, \dots, a_j\}$  thì trả lại  
 $k \in \{i, i+1, \dots, j\}$  sao cho  $x = a_k$ . Ngược lại thì  
trả lại 0

```
1 procedure LinearSearch( $i, j, x$ ):
2   if  $a_i = x$  then // Ở đúng vị trí? Trả lại kết quả
3     return  $i$ 
4   else
5     if  $i = j$  then // Không tìm thấy
6       return 0
7     else // Tìm trong phần còn lại
8       return LinearSearch( $i + 1, j, x$ )
```

### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

#### 11 Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

### Thuật toán tham lam

Giới thiệu

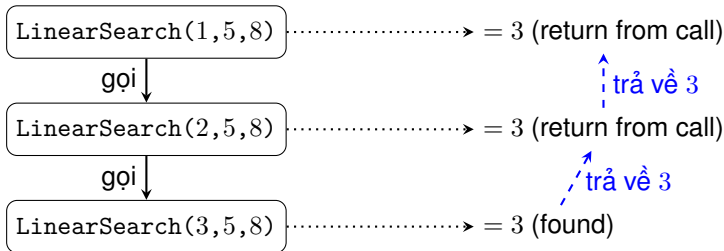
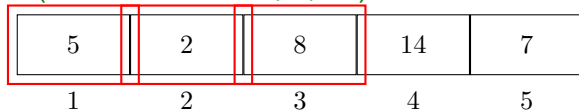
Bài toán Lập lịch

# Thuật toán đệ quy

Tìm kiếm tuyến tính



## Ví dụ 4 (LinearSearch(1, 5, 8))



## Bài tập 2

Với Thuật toán 3, hãy

- Chứng minh tính đúng đắn
- Xây dựng hệ thức truy hồi để đánh giá thời gian chạy

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

12 Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thặng

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

## Tìm kiếm nhị phân



Thuật toán II

Hoàng Anh Đức

### Thuật toán 4: Tìm kiếm nhị phân (Binary Search)

**Input:**  $a_1, a_2, \dots, a_n$ : dãy số nguyên thực sự tăng,  $i, j, x$ : số nguyên,  $1 \leq i \leq j \leq n$

**Output:** Nếu  $x \in \{a_i, a_{i+1}, \dots, a_j\}$  thì trả lại  $k \in \{i, i+1, \dots, j\}$  sao cho  $x = a_k$ . Ngược lại thì trả lại 0

**1 procedure** BinarySearch( $i, j, x$ ):

```
2    $m := \lfloor (i + j) / 2 \rfloor$  // Đi đến giữa dãy
3   if  $x = a_m$  then // Đúng vị trí?
4       return  $m$ 
5   else
6       if  $x < a_m$  và  $i < m$  then //  $x$  ở nửa bên trái?
7           return BinarySearch( $i, m - 1, x$ )
8       else
9           if  $x > a_m$  và  $j > m$  then //  $x$  ở nửa bên phải?
10              return BinarySearch( $m + 1, j, x$ )
11          else // Không tìm thấy
12              return 0
```

#### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

13

37

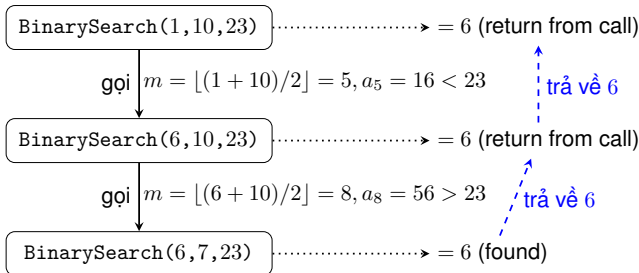
# Thuật toán đệ quy

## Tìm kiếm nhị phân



### Ví dụ 5 (BinarySearch(1, 10, 23))

|   |   |   |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|
| 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 |
| 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 |



$$m = \lfloor (6 + 7)/2 \rfloor = 6, a_6 = 23 = 23$$

## Bài tập 3

Xây dựng hệ thức truy hồi để đánh giá thời gian chạy của Thuật toán 4

### Thuật toán II

Hoàng Anh Đức

#### Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

#### Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

14

37

# Thuật toán đệ quy

## Sắp xếp nổi bọt



Thuật toán II

Hoàng Anh Đức

### Thuật toán 5: Sắp xếp nổi bọt (Bubble Sort)

**Input:**  $A = a_1, a_2, \dots, a_n$ : dãy số nguyên,  $n$ : số nguyên dương

**Output:** Dãy số nguyên sắp thứ tự tăng dần

**procedure** BubbleSort( $A, n$ ):

```
1   if  $n = 1$  then
2       return  $A$  // Trường hợp cơ sở: mảng có 1
3       phần tử đã được sắp xếp
4   for  $i \leftarrow 1$  to  $n - 1$  do
5       if  $a_i > a_{i+1}$  then
6           Hoán đổi giá trị của  $a_i$  và  $a_{i+1}$ 
7   BubbleSort( $A, n - 1$ ) // Đệ quy với phần tử cuối
   cùng đã được đẩy đến đúng vị trí
```

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

15

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

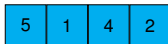
# Thuật toán đệ quy

Sắp xếp nổi bọt



## Ví dụ 6 (BubbleSort([5, 1, 4, 2], 4))

Mảng ban đầu:



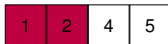
BubbleSort([5, 1, 4, 2], 4)

Lần lượt so sánh: [5, 1], [1, 4], [4, 2]



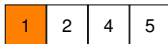
BubbleSort([1, 4, 2, 5], 3)

Lần lượt so sánh: [1, 4], [4, 2]



BubbleSort([1, 2, 4, 5], 2)

Lần lượt so sánh: [1, 2]



BubbleSort([1, 2, 4, 5], 1)

Trường hợp cơ sở:  $n = 1$ , mảng đã sắp xếp

## Bài tập 4

Xây dựng hệ thức truy hồi để đánh giá thời gian chạy của Thuật toán 5

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân

16

Sắp xếp nổi bọt

Sắp xếp chèn  
Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

37

# Thuật toán đệ quy

Sắp xếp chèn



Thuật toán II

Hoàng Anh Đức

## Thuật toán 6: Sắp xếp chèn (Insertion Sort)

**Input:**  $A = a_1, a_2, \dots, a_n$ : dãy số nguyên,  $n$ : số nguyên dương

**Output:** Dãy số nguyên sắp thứ tự tăng dần

```
1 procedure InsertionSort( $A, n$ ):
2   if  $n \leq 1$  then
3     return  $A$  // Trường hợp cơ sở: mảng có 0
                 hoặc 1 phần tử đã được sắp xếp
4   InsertionSort( $A, n - 1$ ) // Sắp xếp đệ quy các
                             phần tử từ  $a_1$  đến  $a_{n-1}$ 
5   // Chèn phần tử  $a_n$  vào vị trí thích hợp trong
     mảng đã sắp xếp
6    $key := a_n$ 
7    $i := n - 1$ 
8   while  $i \geq 1$  và  $a_i > key$  do
9      $a_{i+1} := a_i$ 
10     $i := i - 1$ 
```

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

17

37

# Thuật toán đệ quy

Sắp xếp chèn



## Ví dụ 7 (InsertionSort([5, 2, 4, 1], 4))

Mảng ban đầu:

|   |   |   |   |
|---|---|---|---|
| 5 | 2 | 4 | 1 |
|---|---|---|---|

InsertionSort([5, 2, 4, 1], 4)

gọi ↓

InsertionSort([5, 2, 4, 1], 3)

gọi ↓

InsertionSort([5, 2, 4, 1], 2)

gọi ↓

InsertionSort([5, 2, 4, 1], 1)

Sắp xếp đệ quy [5, 2, 4], sau đó chèn 1

Sắp xếp đệ quy [5, 2], sau đó chèn 4

Sắp xếp đệ quy [5], sau đó chèn 2

Trường hợp cơ sở: mảng [5] đã sắp xếp

|   |   |   |   |
|---|---|---|---|
| 2 | 5 | 4 | 1 |
|---|---|---|---|

Chèn 2 vào [5]

|   |   |   |   |
|---|---|---|---|
| 2 | 4 | 5 | 1 |
|---|---|---|---|

Chèn 4 vào [2, 5]

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 4 | 5 |
|---|---|---|---|

Chèn 1 vào [2, 4, 5]

Kết quả cuối cùng

## Bài tập 5

Xây dựng hệ thức truy hồi để đánh giá thời gian chạy của Thuật toán 6

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thặng dư

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

18

37

# Thuật toán đệ quy

## Sắp xếp trộn



Thuật toán II

Hoàng Anh Đức

### Thuật toán 7: Sắp xếp trộn (Merge Sort)

**Input:**  $L = a_1, a_2, \dots, a_n$ : dãy số nguyên

**Output:** Dãy số nguyên sắp thứ tự tăng dần

**procedure** MergeSort( $L$ ):

**if**  $n > 1$  **then**

$m := \lfloor n/2 \rfloor$

$L_1 := a_1, \dots, a_m; L_2 := a_{m+1}, \dots, a_n$

$L := \text{Merge}(\text{MergeSort}(L_1), \text{MergeSort}(L_2))$

**procedure** Merge( $A, B$ ):

**Input:**  $A = (a_1, \dots, a_{|A|}), B = (b_1, \dots, b_{|B|})$ : dãy số đã sắp thứ tự

**Output:** Dãy các số trong cả  $A$  và  $B$  sắp thứ tự tăng dần

**if**  $A = \emptyset$  **then**

**return**  $B$

**if**  $B = \emptyset$  **then**

**return**  $A$

**if**  $a_1 < b_1$  **then**

**return**  $(a_1, \text{Merge}(a_2, \dots, a_{|A|}, B))$

**else**

**return**  $(b_1, \text{Merge}(A, b_2, \dots, b_{|B|}))$

#### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

19 Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thặng

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

# Thuật toán đệ quy

Sắp xếp trộn



Thuật toán II

Hoàng Anh Đức

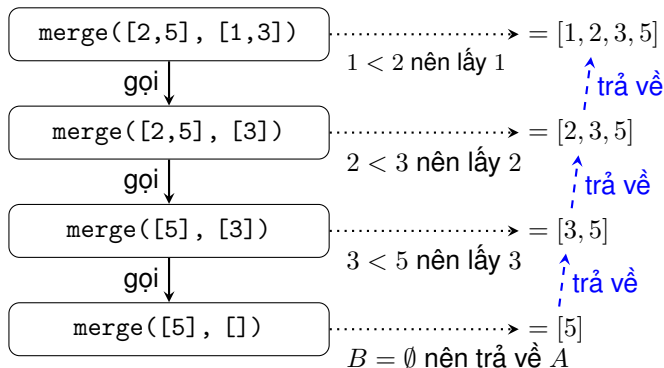
## Ví dụ 8 (Merge([2, 5], [1, 3]))

Mảng A:

|   |   |
|---|---|
| 2 | 5 |
|---|---|

Mảng B:

|   |   |
|---|---|
| 1 | 3 |
|---|---|



Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

20

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

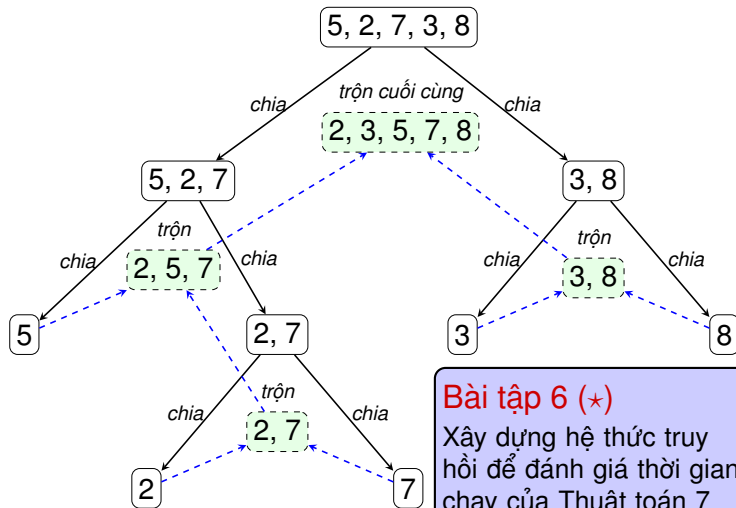
37

# Thuật toán đệ quy

Sắp xếp trộn



Ví dụ 9 (MergeSort([5, 2, 7, 3, 8]))



## Bài tập 6 (\*)

Xây dựng hệ thức truy hồi để đánh giá thời gian chạy của Thuật toán 7

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

21

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

37

# Ước lượng hệ thức truy hồi

Giới thiệu



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

22

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

- Hệ thức truy hồi thường xuất hiện khi phân tích thuật toán đệ quy
- Ước lượng hệ thức truy hồi giúp ta xác định độ phức tạp thời gian của thuật toán
- Hai phương pháp chính để ước lượng hệ thức truy hồi:
  - (1) **Định lý thợ (Master Theorem)**: Công cụ mạnh mẽ giúp ước lượng nhanh chóng các hệ thức có dạng
$$f(n) = af(n/b) + cn^d$$
  - (2) **Cây đệ quy (Recursion Tree)**: Biểu diễn trực quan quá trình tính toán

# Ước lượng hệ thức truy hồi

## Định lý thợ



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

23

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

## Định lý 1: Định lý thợ (Master Theorem)

Gọi  $f$  là một hàm tăng thỏa mãn hệ thức truy hồi

$$f(n) = af(n/b) + cn^d$$

trong đó  $n = b^k$  với  $k$  là số nguyên dương nào đó,  $a \geq 1$ ,  $b$  là số nguyên dương lớn hơn 1, và  $c, d$  là các số thực với  $c$  dương và  $d$  không âm. Ta có

$$f(n) \text{ là } \begin{cases} O(n^d) & \text{nếu } a < b^d \\ O(n^d \log n) & \text{nếu } a = b^d \\ O(n^{\log_b a}) & \text{nếu } a > b^d \end{cases}$$

## Ví dụ 10

- Với  $T(n) = 2T(n/2) + n$ , ta có  $T(n) = O(n \log n)$
- Với  $T(n) = T(n/2) + n$ , ta có  $T(n) = O(n)$
- Với  $T(n) = 3T(n/2) + n$ , ta có  $T(n) = O(n^{\log 3})$

# Ước lượng hệ thức truy hồi

## Định lý thợ



Thuật toán II

Hoàng Anh Đức

## Bài tập 7

Sử dụng Định lý thợ (Định lý 1), hãy ước lượng các hệ thức truy hồi sau theo  $O$ -lớn, giả sử  $T(1) = 1$

(a)  $T(n) = 4T(n/3) + n^2$

(b)  $T(n) = 4T(n/2) + n^2$

(c)  $T(n) = 3T(n/3) + n \log n$

(d)  $T(n) = 3T(n/3) + 1$

(e)  $T(n) = 2T(n/2) + n^n$

(f)  $T(n) = 7T(n/2) + n^2$

(g)  $T(n) = 16T(n/4) + n^2$

(h)  $T(n) = 2T(n/2) - n^2$

(i)  $T(n) = 8T(n/2) + n^3$

(j)  $T(n) = 3T(n/3) + \sqrt{n}$

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

24

37

# Ước lượng hệ thức truy hồi

Cây đệ quy



Thuật toán II

Hoàng Anh Đức

- **Cây đệ quy (Recursion Tree)** là biểu diễn trực quan của quá trình chạy thuật toán đệ quy
- Mỗi nút trong cây biểu diễn **chi phí (cost)** của một **lần gọi đệ quy (recursive call)**
- **Các bước xây dựng và phân tích cây đệ quy:**
  - (1) Vẽ cây với gốc ứng với lần gọi ban đầu  $T(n)$
  - (2) Phân tách mỗi nút thành các chi phí: chi phí không đệ quy (non-recursive cost) và các lần gọi đệ quy (recursive calls)
  - (3) Tiếp tục phân tách cho đến khi đạt trường hợp cơ sở (base case)
  - (4) Tính tổng chi phí theo từng mức (level) của cây
  - (5) Tính tổng chi phí các mức để có kết quả cuối cùng
- **Ưu điểm:** Trực quan, giúp hiểu được bản chất của thuật toán và áp dụng được với nhiều dạng hệ thức mà Định lý thợ không áp dụng được
- **Nhược điểm:** Không phải lúc nào cũng dễ vẽ và phân tích được

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

25

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

37

# Ước lượng hệ thức truy hồi

Cây đệ quy

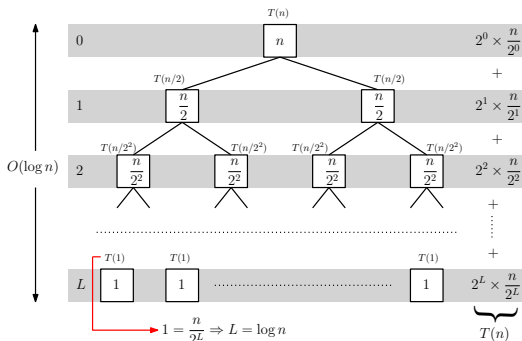


Thuật toán II

Hoàng Anh Đức

## Ví dụ 11

Xét hệ thức truy hồi  $T(n) = 2T(n/2) + n$  với điều kiện ban đầu  $T(1) = 1$  và  $n = 2^k$  với số nguyên  $k \geq 1$  nào đó. Ta vẽ cây đệ quy cho hệ thức này như sau



$$\text{Ta có } T(n) = \sum_{i=0}^L 2^i \cdot \frac{n}{2^i} = n(\log n + 1) = O(n \log n)$$

Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ

26

Cây đệ quy

Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

37

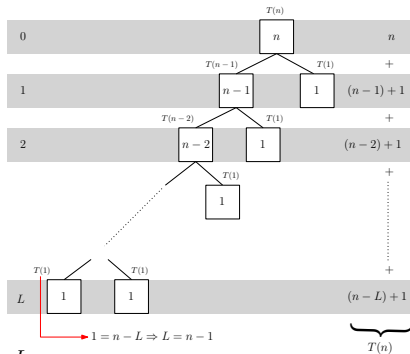
# Ước lượng hệ thức truy hồi

Cây đệ quy



## Ví dụ 12

Xét hệ thức truy hồi  $T(n) = T(n-1) + T(1) + n$  với  $n \geq 1$  và điều kiện ban đầu  $T(1) = 1$ . Ta vẽ cây đệ quy của hệ thức này như sau



$$\text{Ta có } T(n) \leq \sum_{i=0}^L n = O(n^2)$$

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ

Cây đệ quy

Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

27

37

# Ước lượng hệ thức truy hồi

Cây đệ quy



## Bài tập 8

Sử dụng cây đệ quy để ước lượng  $T(n)$  cho bởi các hệ thức truy hồi sau

(a)  $T(n) = 2T(n/2) + n^2$

(b)  $T(n) = T(n/2) + 1$

(c)  $T(n) = 2T(n-1) + 1$

(d) (\*)  $T(n) = 2T(n/2) + n \log n$  (**Đáp án:**  $O(n \log^2 n)$ )

## Bài tập 9

Chứng minh Định lý thợ bằng cách sử dụng cây đệ quy

(a) Vẽ cây đệ quy cho  $T(n) = aT(n/b) + cn^d$  trong đó  $n = b^k$  với  $k$  là số nguyên dương nào đó,  $a \geq 1$ ,  $b$  là số nguyên dương lớn hơn 1, và  $c, d$  là các số thực với  $c$  dương và  $d$  không âm

(b) Tính tổng từng hàng và viết công thức của  $T(n)$  dưới dạng tổng của các hàng trong cây.

(c) Xét các trường hợp  $a < b^d$ ,  $a = b^d$ , và  $a > b^d$

Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

28

Cây đệ quy

Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

37

# Ước lượng hệ thức truy hồi

## Cây đệ quy



Thuật toán II

Hoàng Anh Đức

## Bài tập 10

Xây dựng hệ thức truy hồi cho thời gian chạy của các thuật toán đệ quy đã đề cập trong bài giảng. Giải hoặc ước lượng hệ thức bạn tìm được để chứng minh thời gian chạy của các thuật toán như sau:

- Tính lũy thừa (Thuật toán 2):  $T(n) = O(n)$
- Tìm kiếm tuyến tính (Thuật toán 3):  $T(n) = O(n)$
- Tìm kiếm nhị phân (Thuật toán 4):  $T(n) = O(\log n)$
- Sắp xếp nổi bọt (Thuật toán 5):  $T(n) = O(n^2)$
- Sắp xếp chèn (Thuật toán 6):  $T(n) = O(n^2)$
- Sắp xếp trộn (Thuật toán 7):  $T(n) = O(n \log n)$

Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

29

Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

# Thuật toán tham lam

## Giới thiệu



### Thuật toán II

Hoàng Anh Đức

#### Thuật toán đệ quy

Giới thiệu

Tính giai thừa và lũy thừa

Tìm kiếm tuyến tính

Tìm kiếm nhị phân

Sắp xếp nổi bọt

Sắp xếp chèn

Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu

Định lý thợ

Cây đệ quy

#### Thuật toán tham lam

Giới thiệu

Bài toán Lập lịch

- **Các bài toán tối ưu (optimization problems)** yêu cầu cực đại hóa hoặc cực tiểu hóa một số tham số xét trên tập tất cả các đầu vào có thể
  - Tìm đường đi giữa hai thành phố với khoảng cách **nhỏ nhất**
  - Tìm cách đặt các trạm phát sóng sao cho diện tích phủ sóng **lớn nhất**
- Một **thuật toán tham lam (greedy algorithm)** thường được sử dụng để giải bài toán tối ưu: luôn chọn biện pháp “tốt nhất” ở mỗi bước địa phương (theo một số tiêu chuẩn cục bộ nào đó) với hi vọng sẽ thu được một lời giải tối ưu trên toàn cục
  - Giải thuật này không nhất thiết xuất ra một lời giải tối ưu cho toàn bộ bài toán, nhưng trong nhiều trường hợp cụ thể nó có thể xuất ra lời giải tối ưu
  - Sau khi mô tả cụ thể “lựa chọn tốt nhất ở từng bước”, ta cố gắng chứng minh rằng giải thuật này luôn cho ta một lời giải tối ưu hoặc tìm một phản ví dụ để chỉ ra điều ngược lại

30

37

# Thuật toán tham lam

## Bài toán Lập lịch



### Thuật toán II

Hoàng Anh Đức

#### Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

#### Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

## ■ Bài toán:

### ■ Input:

- Một nhóm các bài giảng với thời gian bắt đầu và kết thúc
- Chỉ có một giảng đường duy nhất
- Khi một bài giảng bắt đầu, nó tiếp diễn cho đến khi kết thúc
- Không có hai bài giảng nào được tiến hành ở cùng thời điểm
- Ngay sau khi một bài giảng kết thúc, một bài giảng khác có thể bắt đầu

### ■ Output: Một danh sách các bài giảng dài nhất có thể

- Ở đây, nếu ta muốn áp dụng giải thuật tham lam, làm thế nào để “lựa chọn tốt nhất” ở mỗi bước của thuật toán? Nói cách khác, ta sẽ chọn bài giảng như thế nào?

- (1) Chọn bài giảng bắt đầu sớm nhất trong số các bài giảng bắt đầu sau các bài giảng ta vừa chọn trước đó?
- (2) Chọn bài giảng ngắn nhất trong số các bài giảng bắt đầu sau các bài giảng ta vừa chọn trước đó?
- (3) Chọn bài giảng kết thúc sớm nhất trong số các bài giảng bắt đầu sau các bài giảng ta vừa chọn trước đó?

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

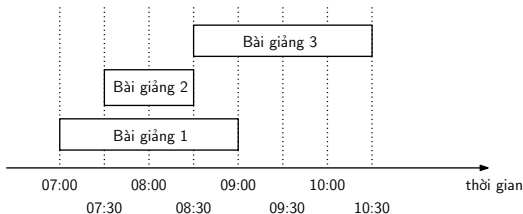
Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

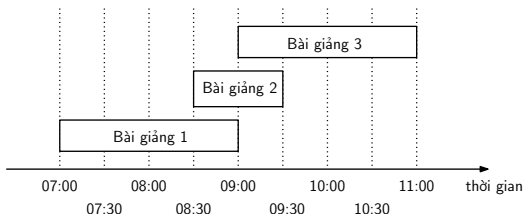
Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

(1) Chọn bài giảng bắt đầu sớm nhất? **X**



(2) Chọn bài giảng ngắn nhất? **X**



(3) Chọn bài giảng kết thúc sớm nhất? **✓**

32

37

# Thuật toán tham lam

## Bài toán Lập lịch



### Thuật toán II

Hoàng Anh Đức

## Thuật toán 8: Lập lịch tham lam (Greedy Scheduling)

**Input:**  $(s_1, e_1), (s_2, e_2), \dots, (s_n, e_n)$ : thời gian bắt đầu và kết thúc bài giảng  $b_1, b_2, \dots, b_n$

**Output:** Danh sách bài giảng  $S$  có số bài giảng lớn nhất trong đó không có hai bài giảng nào xung đột nhau

- 1 Sắp xếp các bài giảng theo thứ tự tăng dần theo thời gian kết thúc và gán lại nhãn bài giảng sao cho

$$e_1 \leq e_2 \leq \dots \leq e_n$$

- 2  $S := \emptyset$

- 3 **for**  $j := 1$  **to**  $n$  **do**

- 4     **if** Bài giảng  $j$  không xung đột với các phần tử của  $S$   
      **then**

- 5          $S := S \cup \{j\}$

- 6 **return**  $S$

### Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

### Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

### Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

33

37

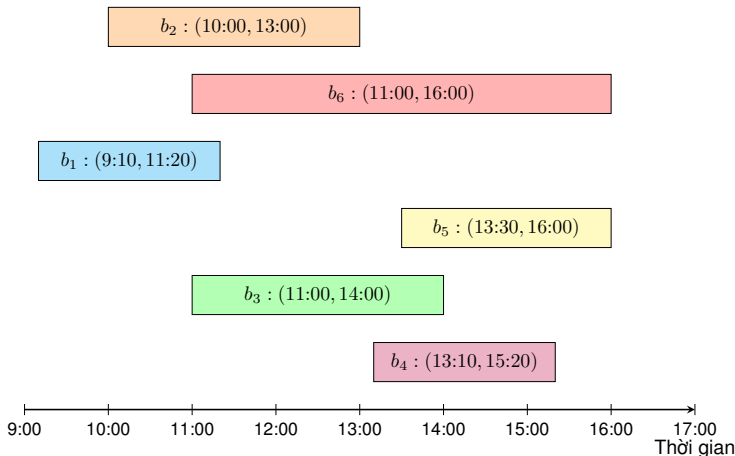
# Thuật toán tham lam

## Bài toán Lập lịch



### Ví dụ 13

Các bài giảng ban đầu:



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch

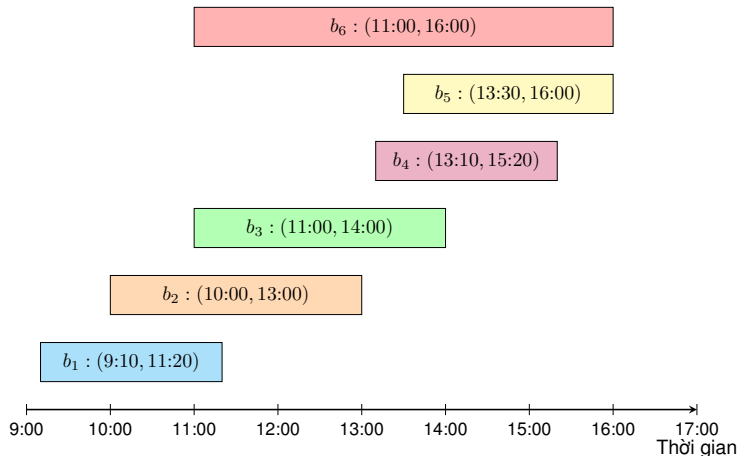


Thuật toán II

Hoàng Anh Đức

## Ví dụ 13

Bước 1: Sắp xếp theo thời gian kết thúc



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

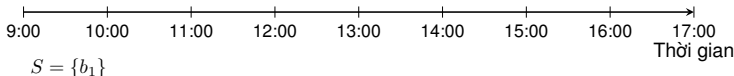
### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 1$ : Thêm  $b_1$  vào  $S$

$b_1 : (9:10, 11:20)$



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

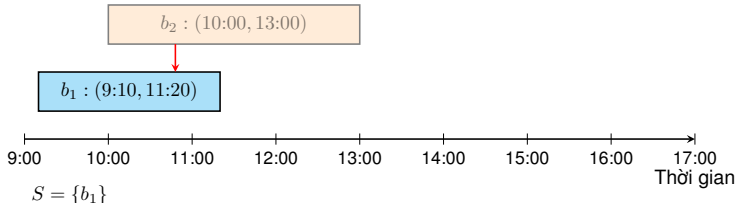
Hoàng Anh Đức

### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 2$ : Không thêm  $b_2$  vì xung đột với  $b_1$



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

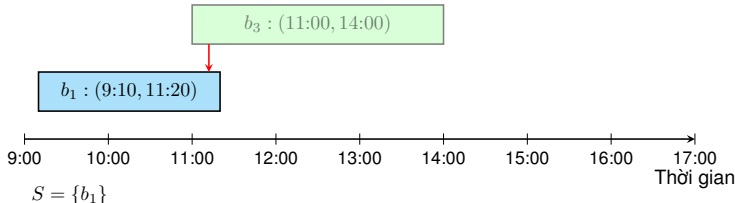
Hoàng Anh Đức

### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 3$ : Không thêm  $b_3$  vì xung đột với  $b_1$



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

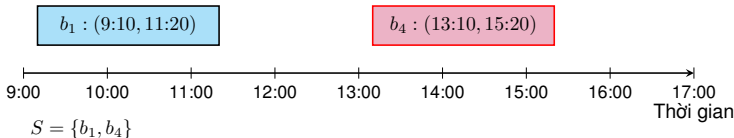
Hoàng Anh Đức

### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 4$ : Thêm  $b_4$  vào  $S$



Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

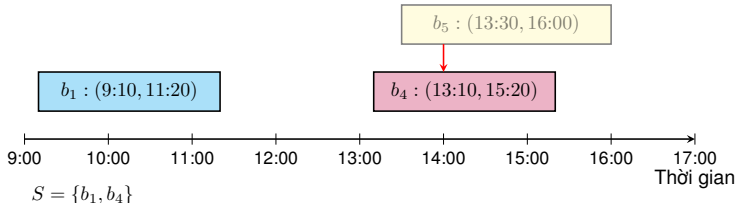
Hoàng Anh Đức

### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 5$ : Không thêm  $b_5$  vì xung đột với  $b_4$



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

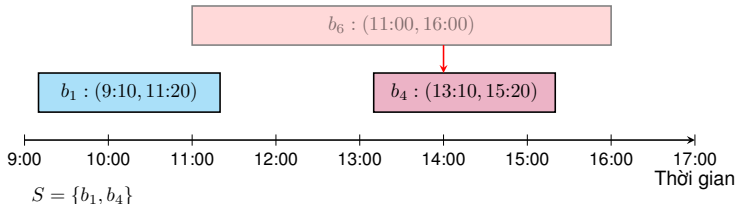
Hoàng Anh Đức

### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Xét  $j = 6$ : Không thêm  $b_6$  vì xung đột với  $b_4$



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

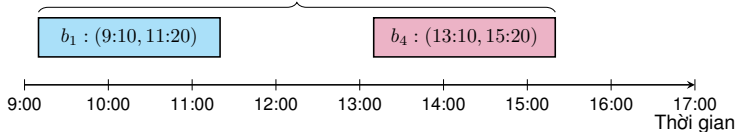
### Ví dụ 13

Bước 2: Khởi tạo  $S = \emptyset$

Bước 3: Duyệt các bài giảng

Kết quả:  $S = \{b_1, b_4\}$

Lịch tối ưu với 2 bài giảng



Thuật toán đệ quy

- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch

34

37

# Thuật toán tham lam

## Bài toán Lập lịch



### Định lý 2

Thuật toán 8 xuất ra một danh sách các bài giảng tối ưu

### Chứng minh.

- Giả sử  $S^* = (x_1, x_2, \dots, x_k)$  là danh sách tối ưu trong đó các bài giảng được sắp xếp theo thứ tự tăng dần theo thời gian kết thúc
- Giả sử  $S = (y_1, y_2, \dots, y_{k'})$  là một danh sách bài giảng xuất ra từ Thuật toán 8 trong đó các bài giảng được sắp xếp theo thứ tự tăng dần theo thời gian kết thúc
- Do  $S^*$  là tối ưu,  $k \geq k'$
- Nếu  $S = S^*$  thì ta có điều phải chứng minh. Ngược lại, nếu  $S \neq S^*$ , gọi  $i$  là chỉ số đầu tiên trong  $\{1, \dots, k'\}$  thỏa mãn  $x_i \neq y_i$ , nghĩa là
$$S^* = \langle x_1 = y_1, x_2 = y_2, \dots, x_{i-1} = y_{i-1}, \mathbf{x_i}, \dots, x_{k'}, \dots, x_k \rangle$$
$$S = \langle x_1 = y_1, x_2 = y_2, \dots, x_{i-1} = y_{i-1}, \mathbf{y_i}, \dots, y_{k'} \rangle$$

#### Thuật toán II

Hoàng Anh Đức

#### Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

#### Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

#### Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

35

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

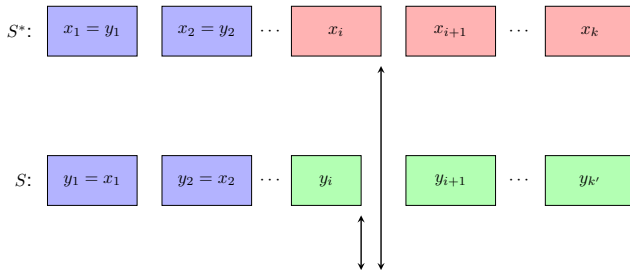
- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch



Nếu không tồn tại  $i \in \{1, \dots, k'\}$  thỏa mãn  $x_i \neq y_i$ , ta có  
 $S^* = \langle x_1 = y_1, x_2 = y_2, \dots, x_{k'} = y_{k'}, x_{k'+1}, \dots, x_k \rangle$   
Ngược lại, thuật toán tham lam chọn  $y_i$  vì nó kết thúc sớm hơn  $x_i$

36

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

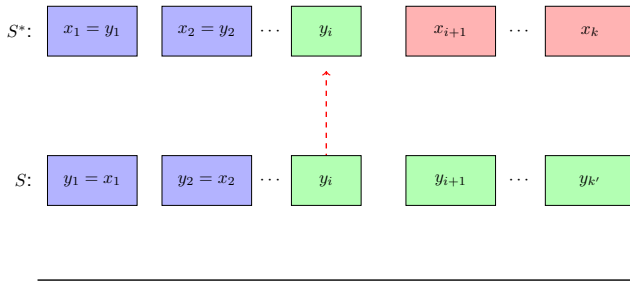
- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch



Thay thế  $x_i$  bằng  $y_i$  trong  $S^*$  vẫn giữ tính tối ưu  
vì  $y_i$  kết thúc sớm hơn và không xung đột với  $x_{i+1}, \dots, x_k$

36

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

### Thuật toán đệ quy

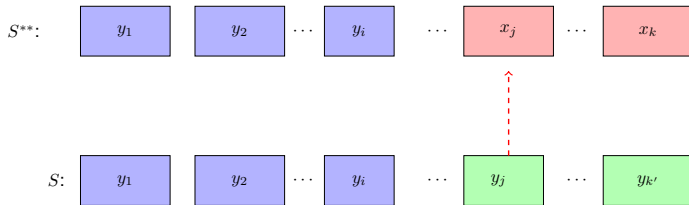
- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

### Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

### Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch



Thời gian

Sau khi thay thế  $x_i$  bằng  $y_i$ , ta thu được lời giải tối ưu mới  $S^{**}$ .  
Tiếp đó, tìm  $j$  là chỉ số nhỏ nhất thỏa mãn  $x_j \neq y_j$ .  
Lý luận tương tự để thay thế  $x_j$  bằng  $y_j$ .

36

37

# Thuật toán tham lam

## Bài toán Lập lịch



Thuật toán II

Hoàng Anh Đức

Thuật toán đệ quy

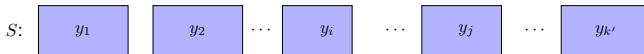
- Giới thiệu
- Tính giai thừa và lũy thừa
- Tìm kiếm tuyến tính
- Tìm kiếm nhị phân
- Sắp xếp nổi bọt
- Sắp xếp chèn
- Sắp xếp trộn

Ước lượng hệ thức truy hồi

- Giới thiệu
- Định lý thợ
- Cây đệ quy

Thuật toán tham lam

- Giới thiệu
- Bài toán Lập lịch



Thời gian

Tiếp tục quá trình này cho đến khi  $S^* = \langle y_1, y_2, \dots, y_{k'}, x_{k'+1}, \dots, x_k \rangle$   
Do thuật toán kết thúc khi chọn  $y_{k'}$ ,  
các phần tử  $x_{k'+1}, \dots, x_k$  phải xung đột với  $y_{k'}$  nên không thể tồn tại  
Do đó  $S^* = S = \langle y_1, y_2, \dots, y_{k'} \rangle$  là tối ưu

36

37

# Thuật toán tham lam

## Bài toán Lập lịch



### Thuật toán II

Hoàng Anh Đức

### Định lý 3

*Thuật toán 8 chạy trong thời gian  $O(n \log n)$ , với  $n$  là số lượng bài giảng*

## Chứng minh.

Phân tích độ phức tạp:

- Sắp xếp  $n$  bài giảng theo thời gian kết thúc:  $O(n \log n)$ 
  - Ví dụ, sử dụng Thuật toán 7 (Sắp xếp trộn)
- Vòng lặp **for** duyệt qua  $n$  bài giảng:  $O(n)$
- Kiểm tra xung đột giữa bài giảng  $j$  với các bài giảng trong  $S$ :
  - Phương pháp đơn giản:  $O(|S|) = O(n)$  trong trường hợp xấu nhất
  - Phương pháp tối ưu: Chỉ cần kiểm tra thời gian bắt đầu của bài giảng  $j$  với thời gian kết thúc của bài giảng cuối cùng trong  $S$ :  $O(1)$

### Thuật toán đệ quy

Giới thiệu  
Tính giai thừa và lũy thừa  
Tìm kiếm tuyến tính  
Tìm kiếm nhị phân  
Sắp xếp nổi bọt  
Sắp xếp chèn  
Sắp xếp trộn

### Ước lượng hệ thức truy hồi

Giới thiệu  
Định lý thợ  
Cây đệ quy

### Thuật toán tham lam

Giới thiệu  
Bài toán Lập lịch

37

37

Part I

Phụ lục

# Nội dung



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

Định lý thợ

Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Định lý thợ

Phát biểu Định lý



Thuật toán II

Hoàng Anh Đức

## 2 Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

### Định lý: (Nhắc lại Định lý 1)

Gọi  $f$  là một hàm tăng thỏa mãn hệ thức truy hồi

$$f(n) = af(n/b) + cn^d$$

trong đó  $n = b^k$  với  $k$  là số nguyên dương nào đó,  $a \geq 1$ ,  $b$  là số nguyên dương lớn hơn 1, và  $c, d$  là các số thực với  $c$  dương và  $d$  không âm. Ta có

$$f(n) \text{ là } \begin{cases} O(n^d) & \text{nếu } a < b^d \\ O(n^d \log n) & \text{nếu } a = b^d \\ O(n^{\log_b a}) & \text{nếu } a > b^d \end{cases}$$



Ta có

$$\begin{aligned}f(n) &= af(n/b) + cn^d \\&= a(af(n/b^2) + c(n/b)^d) + cn^d \\&= a^2 f(n/b^2) + cn^d(1 + \frac{a}{b^d}) \\&= a^2 (af(n/b^3) + c(n/b^2)^d) + cn^d(1 + \frac{a}{b^d}) \\&= a^3 f(n/b^3) + cn^d(1 + \frac{a}{b^d} + \left(\frac{a}{b^d}\right)^2) \\&= \dots \\&= a^k f(n/b^k) + cn^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i\end{aligned}$$

3 Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)



Xét phương trình  $f(n) = a^k f(n/b^k) + cn^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i$

(1) Nếu  $a = b^d$ , ta có  $\sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i = k$

(2) Nếu  $a < b^d$ , ta có  $\sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i \leq \sum_{i=0}^{\infty} \left(\frac{a}{b^d}\right)^i = \frac{1}{1 - \frac{a}{b^d}}$ . Do

$a, b, d$  là các hằng số, ta có  $\sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i = O(1)$

(3) Nếu  $a > b^d$ , ta có  $\sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i = \frac{(a/b^d)^k - 1}{a/b^d - 1} = O((a/b^d)^k)$

#### 4 Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

Do đó,

$$f(n) = a^k f(n/b^k) + cn^d \cdot \begin{cases} O(k) & \text{nếu } a = b^d \\ O(1) & \text{nếu } a < b^d \\ O((a/b^d)^k) & \text{nếu } a > b^d \end{cases}$$

Từ  $n = b^k$ , ta có  $k = \log_b n$ . Do đó,

■ Nếu  $a = b^d$ , ta có  $f(n) = O(a^{\log_b n}) + O(n^d \log_b n) = O(n^{\log_b a}) + O(n^d \log_b n) = O(n^d \log n)$ . (Chú ý là khi  $a \leq b^d$ , ta có  $n^{\log_b a} \leq n^d$ )

■ Nếu  $a < b^d$ , ta có  $f(n) = O(n^d) + O(n^d) = O(n^d)$ . (Chú ý là khi  $a \leq b^d$ , ta có  $n^{\log_b a} \leq n^d$ )

■ Nếu  $a > b^d$ , ta có

$$f(n) = O(a^{\log_b n}) + O(a^{\log_b n}) = O(a^{\log_b n}) = O(n^{\log_b a}).$$

$$\begin{aligned} \text{(Chú ý là khi } a > b^d, \text{ ta có } (a/b^d)^{\log_b n} &= (b^{\log_b(a/b^d)})^{\log_b n} = \\ &= (b^{\log_b a - d})^{\log_b n} = n^{\log_b(a/b^d)} = n^{\log_b a - d} = O(n^{\log_b a}) \end{aligned}$$

5

## Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)



## Định lý 4: Định lý thợ

Gọi  $f$  là một hàm tăng thỏa mãn hệ thức truy hồi

$$f(n) = af(n/b) + g(n)$$

trong đó  $a \geq 1$ ,  $b > 1$  là các hằng số nào đó, và  $g(n)$  là một hàm thỏa mãn  $g(n) \geq 0$  khi  $n$  đủ lớn. Ta có

- (1)  $f(n)$  là  $\Theta(g(n))$  nếu  $g(n) = \Omega(n^{\log_a b + \epsilon})$  với  $\epsilon > 0$  và  $g(n)$  thỏa mãn điều kiện  $ag(n/b) \leq cg(n)$  với  $n$  đủ lớn và  $c < 1$  là hằng số dương nào đó
- (2)  $f(n)$  là  $\Theta(n^{\log_b a} \log^{k+1} n)$  nếu  $g(n) = \Theta(n^{\log_b a} \log^k n)$  với  $k \geq 0$  nào đó. (Trong phần lớn các trường hợp thì  $k = 0$ .)
- (3)  $f(n)$  là  $\Theta(n^{\log_b a})$  nếu  $g(n) = O(n^{\log_b a - \epsilon})$  với  $\epsilon > 0$

6

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Có thể bỏ qua sàn và trần



Thuật toán II

Hoàng Anh Đức

Định lý thợ

7 Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Chú ý

Có thể bỏ qua sàn và trần khi phân tích độ phức tạp của thuật toán

## Ví dụ 14 (Thời gian chạy của thuật toán sắp xếp trộn)

- Nếu bạn phân tích giả mã của thuật toán sắp xếp trộn (Thuật toán 7), bạn sẽ nhận ra rằng thời gian chạy của thuật toán tuân theo hệ thức:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + O(n) \quad (1)$$

(Xem Bài tập 6)

- Tuy nhiên, trong nhiều tài liệu, bạn sẽ thấy đề cập rằng thời gian chạy của thuật toán tuân theo hệ thức

$$T(n) = 2T(n/2) + O(n) \quad (2)$$

# Có thể bỏ qua sàn và trần



Thuật toán II

Hoàng Anh Đức

Định lý thợ

8 Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 14 (Thời gian chạy của thuật toán sắp xếp trộn)

Cả hai hệ thức trên đều đúng. Lý do ở đây là ta có thể bỏ qua sàn và trần bằng cách lý luận như sau:

- Đầu tiên, do chúng ta chỉ quan tâm đến chặn trên (vì ta luôn đánh giá theo  $O$ -lớn), ta có thể đánh giá

$$T(n) \leq 2T(\lceil n/2 \rceil) + O(n) \leq 2T(n/2 + 1) + O(n)$$

- Tiếp đó, ta định nghĩa một hàm mới  $S(n) = T(n + \alpha)$  bằng cách chọn giá trị của  $\alpha$  sao cho  $S(n) \leq 2S(n/2) + O(n)$ . Ta có

$$S(n) = T(n + \alpha)$$

định nghĩa của  $S$

$$\leq 2T(n/2 + \alpha/2 + 1) + O(n + \alpha)$$

đánh giá cho  $T$

$$= 2S(n/2 - \alpha/2 + 1) + O(n + \alpha)$$

định nghĩa của  $S$

Chọn  $\alpha = 2$ , ta có  $S(n) \leq 2S(n/2) + O(n)$

- Ước lượng  $S(n)$  (ví dụ, bằng Định lý thợ), ta có  $S(n) = O(n \log n)$ , và do đó  $T(n - 2) = O((n - 2) \log(n - 2)) = O(n \log n)$

# Có thể bỏ qua sàn và trần



Thuật toán II

Hoàng Anh Đức

Định lý thợ

9 Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Bài tập 11

Xấp xỉ hệ thức truy hồi  $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n - 1$  với điều kiện ban đầu  $T(2) = 1$  và  $T(1) = 0$ . (**Chú ý:**  $T(n)$  là thời gian chạy của thuật toán sắp xếp trộn)

(**Gợi ý:** Với  $n = 2^k$  ( $k \in \mathbb{N}$ ), chứng minh  $T(n) = n \log n - n + 1$ . Sử dụng quy nạp, chứng minh với mọi  $n \in \mathbb{Z}^+$ ,  $T(n) \approx n \log n - n + 1$ .)

## Bài tập 12

*Cây van Emde Boas (van Emde Boas tree, vEB tree, hoặc van Emde Boas priority queue)* là một cấu trúc dữ liệu đệ quy cho phép chúng ta chèn, xóa và tìm kiếm các khóa từ một tập hợp  $U = \{1, 2, \dots, n\}$  một cách nhanh chóng. (Nó giải quyết cùng một vấn đề như cây tìm kiếm nhị phân, nhưng thời gian chạy sẽ tính theo kích thước của tập vũ trụ  $U$  thay vì số lượng khóa được lưu trữ.) Cây van Emde Boas có thời gian chạy cho bởi

$T(n) = T(\sqrt{n}) + 1$  và  $T(1) = 1$ . Hãy giải hệ thức truy hồi này. (**Gợi ý:** Đặt  $S(k) = T(2^k)$ . **Đáp án:**  $T(n) = O(\log \log n)$ )

## Bài tập 13

Ước lượng các hệ thức truy hồi sau:

(a)  $T(n) = T(\lceil n/3 \rceil) + T(\lfloor 2n/3 \rfloor) + O(n)$

(b)  $(\star) T(n) = \sqrt{n}T(\lceil \sqrt{n} \rceil) + O(n)$  (**Gợi ý:** Đặt  $S(k) = T(2^{2^k})$ ). **Đáp án:**  
 $T(n) = O(n \log \log n)$ )

# Có thể bỏ qua sàn và trần



Thuật toán II

Hoàng Anh Đức

Định lý thợ

10 Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Bài tập 14 (★)

Cho một mảng đã sắp xếp  $A[1 \dots n]$ , và ta cần tìm chỉ số  $i$  thỏa mãn  $A[i-1] < x \leq A[i]$ . (Bài toán này có thể giải được bằng thuật toán tìm kiếm nhị phân.) Dưới đây là mô tả thuật toán ROOTSEARCH để giải quyết bài toán này:

- Nếu  $n$  nhỏ (giả sử, nhỏ hơn 100), tìm chỉ số bằng thuật toán tìm kiếm tuyến tính. Ngược lại:
- Định nghĩa mileposts  $:= A[\sqrt{n}], A[2\sqrt{n}], A[3\sqrt{n}], \dots, A[n]$  là danh sách các phần tử cách đều nhau  $\sqrt{n}$  phần tử trong  $A$ .
- Đệ quy, tìm post  $:= \text{ROOTSEARCH}(\text{mileposts}, x)$ .
- Trả về  $\text{ROOTSEARCH}(A[(\text{post} - 1)\sqrt{n}, \dots, \text{post}\sqrt{n}], x)$ .

# Có thể bỏ qua sàn và trần



## Bài tập 14 (tiếp tục)

Thuật toán II

Hoàng Anh Đức

Định lý thợ

11 Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

ROOTSEARCH( $A[1 \dots n], x$ ):

if  $n < 100$  then *⟨⟨Trường hợp cơ sở: mảng nhỏ⟩⟩*

*⟨⟨Tìm kiếm tuyến tính⟩⟩*

for  $i := 1$  to  $n$  do

if  $A[i - 1] < x \leq A[i]$  then

return  $i$

*⟨⟨Định nghĩa các cột mốc⟩⟩*

$k := \lfloor \sqrt{n} \rfloor$

mileposts := new array of size  $\lceil n/k \rceil$

for  $i := 1$  to  $\lceil n/k \rceil$  do

mileposts[ $i$ ] :=  $A[\min(i \cdot k, n)]$

*⟨⟨Tìm cột mốc thích hợp⟩⟩*

post := ROOTSEARCH(mileposts,  $x$ )

*⟨⟨Tìm kiếm trong đoạn con đã xác định⟩⟩*

return ROOTSEARCH( $A[(post - 1) \cdot k + 1 \dots \min(post \cdot k, n)], x$ )

Hãy xác định hệ thức truy hồi biểu diễn thời gian chạy của thuật toán ROOTSEARCH và ước lượng hệ thức bạn tìm được

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



**Bài toán:** Tính số Fibonacci thứ  $n$ , được định nghĩa như sau:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \quad \text{với } n \geq 2$$

**Thuật toán 9:** Thuật toán tính số Fibonacci thứ  $n$

**Input:** Số nguyên không âm  $n$

**Output:** Số Fibonacci thứ  $n$

```
1 procedure Fibonacci( $n$ ):  
2   if  $n = 0$  then  
3     return 0  
4   else  
5     if  $n = 1$  then  
6       return 1  
7     else  
8       return Fibonacci( $n - 1$ ) +  
         Fibonacci( $n - 2$ )
```

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và  
trần

Một số thuật toán đệ  
quy khác

12

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Ví dụ 15 (Fibonacci(4))

Fibonacci(4)

Tính  $F_4$  bằng đệ quy

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

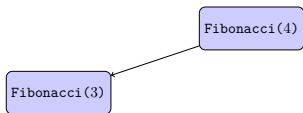
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



Đầu tiên, tính Fibonacci(3)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

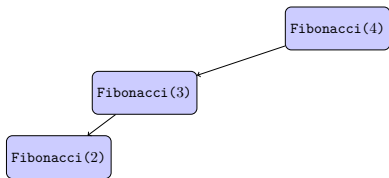
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



Để tính Fibonacci(3), ta cần tính Fibonacci(2)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

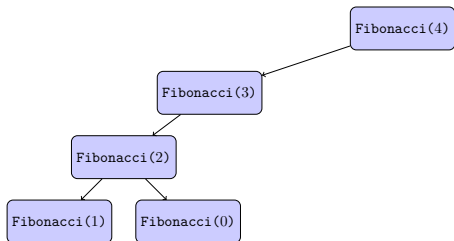
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



Để tính Fibonacci(2), ta cần Fibonacci(1) và Fibonacci(0)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

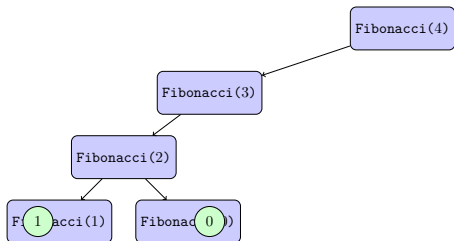
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



Trường hợp cơ sở:  $\text{Fibonacci}(1) = 1$  và  $\text{Fibonacci}(0) = 0$

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

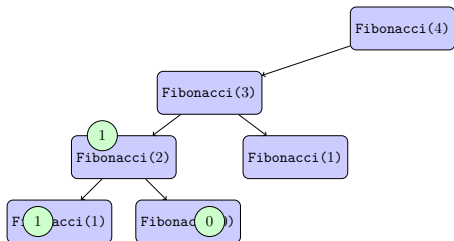
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



$$\text{Fibonacci}(2) = \text{Fibonacci}(1) + \text{Fibonacci}(0) = 1 + 0 = 1$$

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

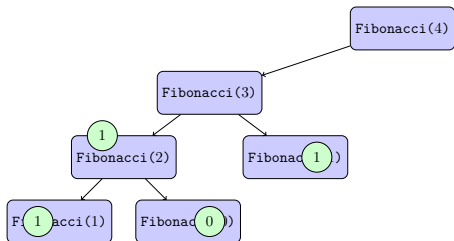
13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Ví dụ 15 (Fibonacci(4))



Với Fibonacci(3), ta cũng cần tính  $\text{Fibonacci}(1) = 1$

# Một số thuật toán đệ quy khác

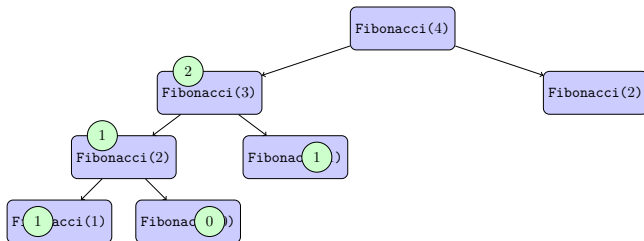
Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Ví dụ 15 (Fibonacci(4))



$$\text{Fibonacci}(3) = \text{Fibonacci}(2) + \text{Fibonacci}(1) = 1 + 1 = 2$$

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

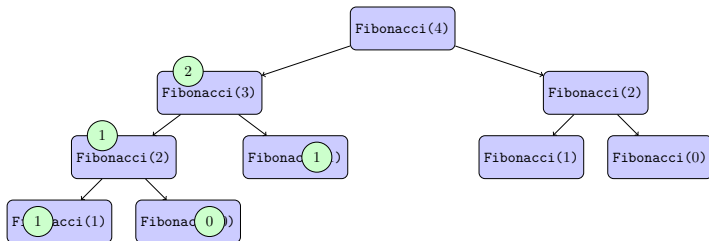
Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



## Ví dụ 15 (Fibonacci(4))



Bây giờ cho phần thứ hai của Fibonacci(4), tính Fibonacci(2)

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

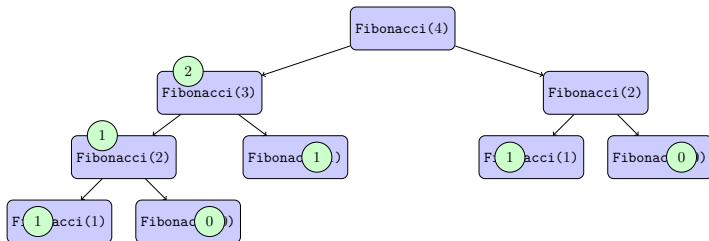
Tìm số Fibonacci thứ  $n$



Thuật toán II

Hoàng Anh Đức

Ví dụ 15 (Fibonacci(4))



$$\text{Fibonacci}(2) = \text{Fibonacci}(1) + \text{Fibonacci}(0) = 1 + 0 = 1$$

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

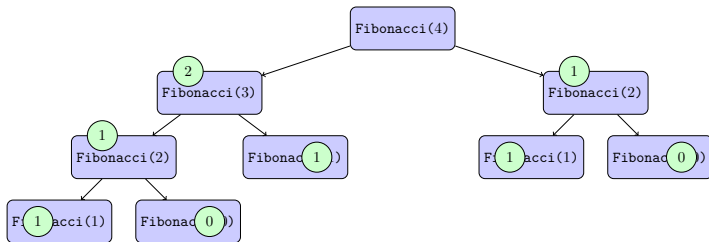
Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



## Ví dụ 15 (Fibonacci(4))



Cuối cùng,  $\text{Fibonacci}(4) = \text{Fibonacci}(3) + \text{Fibonacci}(2) = 2 + 1 = 3$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và  
trần

Một số thuật toán đệ  
quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

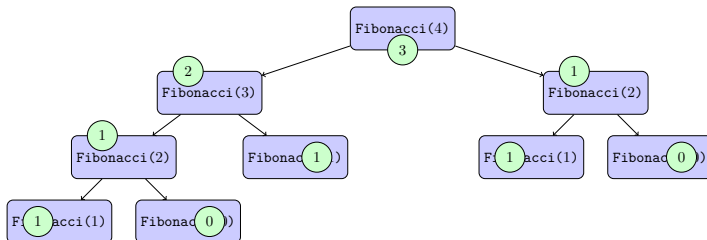
Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



## Ví dụ 15 (Fibonacci(4))



Cuối cùng,  $\text{Fibonacci}(4) = \text{Fibonacci}(3) + \text{Fibonacci}(2) = 2 + 1 = 3$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

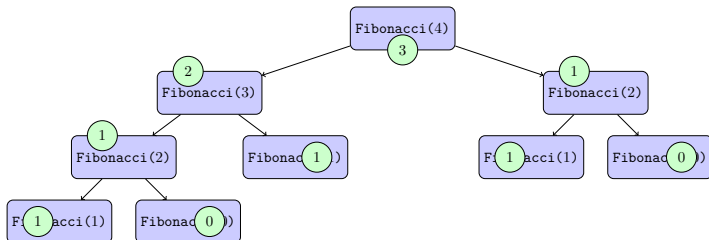
Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$



## Ví dụ 15 (Fibonacci(4))



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

13

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Bài tập 15

- (a) Chứng minh tính đúng đắn của thuật toán tìm số Fibonacci đệ quy
- (b) Chứng minh rằng thời gian chạy của thuật toán là  $O(2^n)$
- (c) (★) Cải tiến thuật toán để giảm độ phức tạp thời gian

# Một số thuật toán đệ quy khác

Tháp Hà Nội



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

**Bài toán:** Di chuyển  $n$  đĩa từ cọc A sang cọc C với sự trợ giúp của cọc B, trong đó không có hai đĩa nào có cùng kích thước, tuân theo các quy tắc:

- Chỉ được di chuyển một đĩa mỗi lượt
- Không được đặt đĩa lớn hơn lên trên đĩa nhỏ hơn

---

**Thuật toán 10:** Thuật toán giải bài toán Tháp Hà Nội

---

**Input:** Số nguyên  $n$ , cọc nguồn A, cọc đích C, cọc trung gian B

**Output:** Dãy các bước di chuyển

**procedure** Hanoi( $n, A, C, B$ ):

```
1   if  $n = 1$  then
2       Di chuyển đĩa trên cùng từ A sang C
3       return
4
5   Hanoi( $n - 1, A, B, C$ )
6   Di chuyển đĩa trên cùng từ A sang C
7   Hanoi( $n - 1, B, C, A$ )
```

14

19

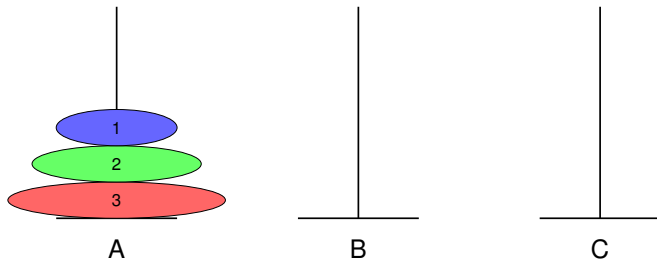
# Một số thuật toán đệ quy khác

Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Trạng thái ban đầu: tất cả đĩa ở cọc A



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

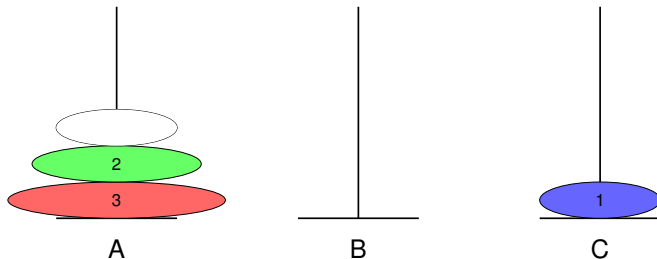
Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 1: Di chuyển đĩa 1 từ A sang C

Hanoi(1, A, C, B)



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

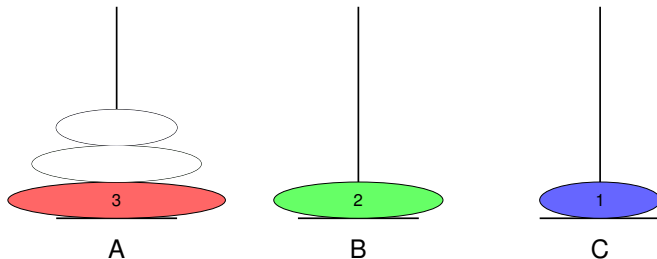
Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 2: Di chuyển đĩa 2 từ A sang B

Đang thực hiện `Hanoi(2, A, B, C)`



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

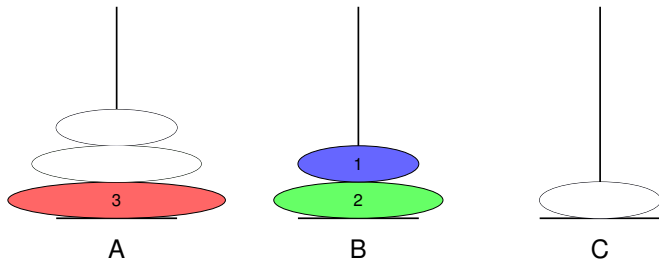
Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 3: Di chuyển đĩa 1 từ C sang B

Hanoi(1, C, B, A)



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

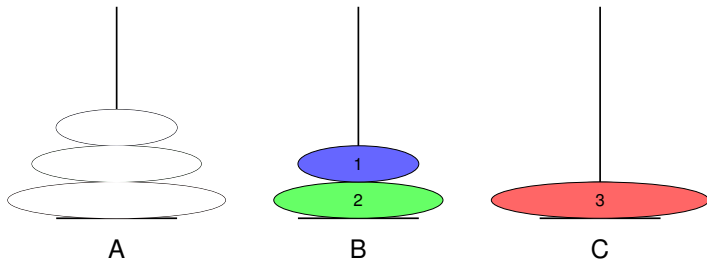
# Một số thuật toán đệ quy khác

Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 4: Di chuyển đĩa 3 từ A sang C



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

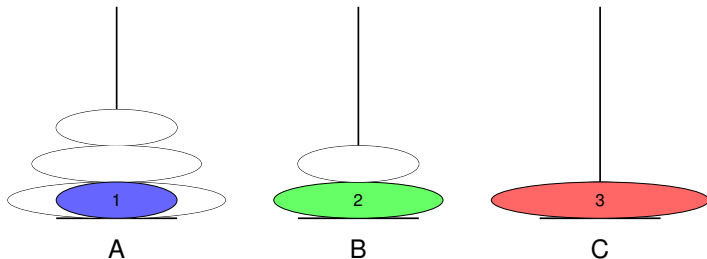
Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 5: Di chuyển đĩa 1 từ B sang A

Hanoi(1, B, A, C)



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

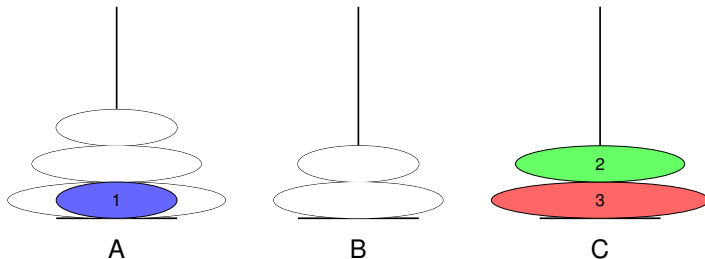
Tháp Hà Nội



## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 6: Di chuyển đĩa 2 từ B sang C

Đang thực hiện Hanoi(2, B, C, A)



## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

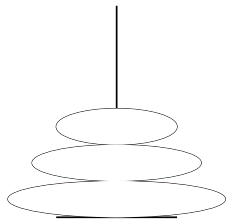
Tháp Hà Nội



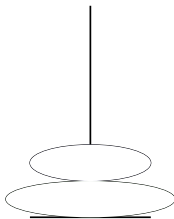
## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Bước 7: Di chuyển đĩa 1 từ A sang C

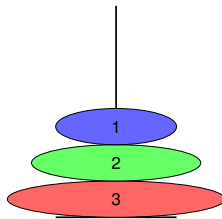
Hanoi(1, A, C, B)



A



B



C

## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

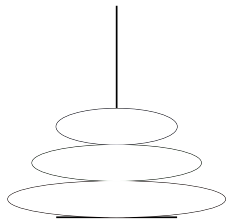
Tháp Hà Nội



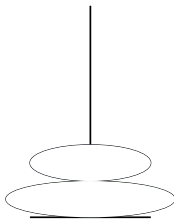
## Ví dụ 16 (Minh họa giải thuật Tháp Hà Nội với 3 đĩa)

Hoàn thành! Tất cả các đĩa đã được chuyển từ cọc A sang cọc C

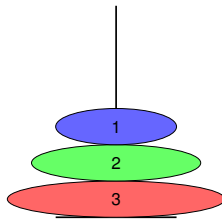
Total:  $2^3 - 1 = 7$  bước



A



B



C

## Bài tập 16

- Chứng minh tính đúng đắn của thuật toán giải bài toán Tháp Hà Nội
- Chứng minh rằng số bước di chuyển tối thiểu để giải quyết bài toán với  $n$  đĩa là  $2^n - 1$

Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sàn và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

15

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

# Một số thuật toán đệ quy khác

Sắp xếp nhanh (Quicksort)



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

**Bài toán:** Sắp xếp một mảng các phần tử theo thứ tự tăng dần.  
**Ý tưởng:**

- Chọn một phần tử trong mảng làm **chốt (pivot)**
- Phân hoạch mảng thành hai phần: các phần tử nhỏ hơn pivot và các phần tử lớn hơn pivot
- Đặt pivot vào đúng vị trí của nó
- Đệ quy sắp xếp hai phần trước và sau pivot

16

19

# Một số thuật toán đệ quy khác

Sắp xếp nhanh (Quicksort)



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sản và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Thuật toán 11: Thuật toán sắp xếp nhanh (Quicksort)

**Input:** Mảng  $A$  và các chỉ số  $p$  và  $r$

**Output:** Mảng  $A$  được sắp xếp trong đoạn từ  $p$  đến  $r$

```
1 procedure QuickSort( $A, p, r$ ):
2     // Sắp xếp mảng  $A$  từ vị trí  $p$  đến  $r$ 
3     if  $p < r$  then
4         // Điều kiện dừng: nếu mảng có ít nhất 2 phần tử
5          $q \leftarrow \text{Partition}(A, p, r)$            //  $q$  là vị trí cuối cùng của pivot
6         QuickSort( $A, p, q - 1$ )           // Đệ quy sắp xếp mảng con bên trái pivot
7         QuickSort( $A, q + 1, r$ )           // Đệ quy sắp xếp mảng con bên phải pivot

8 procedure Partition( $A, p, r$ ):
9     // Phân hoạch mảng và trả về vị trí của pivot
10     $x \leftarrow A[r]$            // Chọn pivot là phần tử cuối cùng của mảng
11     $i \leftarrow p - 1$            //  $i$  theo dõi vị trí cuối cùng của vùng nhỏ hơn pivot
12    for  $j \leftarrow p$  to  $r - 1$  do
13        // Duyệt qua tất cả các phần tử trừ pivot
14        if  $A[j] \leq x$  then
15            // /* Nếu phần tử hiện tại không lớn hơn pivot */
16             $i \leftarrow i + 1$            // Mở rộng vùng các phần tử nhỏ hơn pivot
17            Đổi chỗ  $A[i]$  và  $A[j]$            // Đưa phần tử nhỏ hơn pivot về đầu mảng

18    Đổi chỗ  $A[i + 1]$  và  $A[r]$            // Đặt pivot vào đúng vị trí của nó
19    return  $i + 1$            // Trả về vị trí của pivot trong mảng đã phân hoạch
```

17

19

# Một số thuật toán đệ quy khác

Sắp xếp nhanh (Quicksort)



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

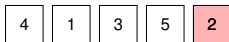
Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

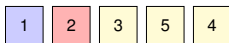
Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

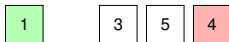
## Ví dụ 17 (Minh họa thuật toán Quicksort)



Mảng ban đầu, pivot = 2

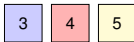


Sau phân hoạch: [1] pivot=2 [3,5,4]



Phần bên trái chỉ có 1 phần tử, đã sắp xếp

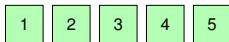
Phân hoạch phần bên phải, pivot = 4



Sau phân hoạch: [3] pivot=4 [5]



Phần bên phải đã sắp xếp



Mảng đã được sắp xếp hoàn chỉnh

18

19

# Một số thuật toán đệ quy khác

Sắp xếp nhanh (Quicksort)



Thuật toán II

Hoàng Anh Đức

Định lý thợ

Có thể bỏ qua sân và trần

Một số thuật toán đệ quy khác

Tìm số Fibonacci thứ  $n$

Tháp Hà Nội

Sắp xếp nhanh (Quicksort)

## Bài tập 17

- (a) Chứng minh tính đúng đắn của thuật toán sắp xếp nhanh
- (b) Phân tích độ phức tạp thời gian của thuật toán sắp xếp nhanh:
  - Trường hợp tốt nhất:  $O(n \log n)$
  - Trường hợp xấu nhất:  $O(n^2)$

19

19